

Product Requirements Document (PRD)

AI Interview Platform (v1)

Version: 1.0

Status: Draft

Owner: Backend Engineering

1. Vision

Develop an AI-powered mock interview platform that enables users to practice interviews tailored to a specific job description using their own resume.

The platform should simulate a realistic technical interview through voice interaction, dynamically adapt questions based on previous responses, and provide a structured performance report highlighting strengths and areas for improvement.

The primary objective of Version 1 is to deliver a seamless interview experience with minimal user friction while maintaining a backend architecture that is scalable, maintainable, and production-ready.

2. Goals

Primary Goals

- Conduct AI-driven mock interviews.
- Personalize interviews using Resume + Job Description.
- Generate adaptive interview questions.
- Evaluate user responses.
- Produce an actionable interview report.

Secondary Goals

- Build a production-quality backend architecture.
 - Keep frontend intentionally minimal.
 - Design APIs that remain stable as the product grows.
-

3. Target Users

- College students preparing for placements.
- Software engineers preparing for interviews.

- Professionals switching companies.
 - Users wanting realistic interview practice.
-

4. User Flow

Landing Page



Upload Resume



Paste / Upload Job Description



(Optional)

Interview Duration

Difficulty



Create Interview Session



Resume Parsing



Job Description Parsing



Candidate Profile Generation



Interview Blueprint Generation



Interview Starts

↓

Adaptive Questioning

↓

Interview Ends

↓

Performance Evaluation

↓

Generate Report

↓

Download Report

5. Functional Requirements

5.1 Resume Upload

Accept:

- PDF
- DOCX

Extract:

- Skills
- Experience
- Projects
- Education
- Certifications

Store both:

- Original document
 - Parsed representation
-

5.2 Job Description Upload

Accept:

- Text
- PDF

Extract:

- Role
 - Required skills
 - Preferred skills
 - Experience
 - Responsibilities
-

5.3 Candidate Profile Generation

Generate a structured profile from the resume.

Example:

- Technical Skills
 - Experience Level
 - Primary Domains
 - Project History
-

5.4 Job Profile Generation

Generate a structured representation of the job.

Example:

- Role
 - Skill Requirements
 - Seniority
 - Expected Knowledge Areas
-

5.5 Interview Blueprint

Generate an interview strategy instead of a fixed list of questions.

The blueprint defines:

- Interview sections
- Topics

- Difficulty
 - Number of questions
 - Evaluation objectives
 - Topic weightage
-

5.6 Interview Session

The interview consists of multiple conversational rounds.

Each round:

AI Question

↓

User Answer

↓

Evaluation

↓

Adaptive Next Question

Questions should adapt based on previous responses.

5.7 Conversation Storage

Store every interaction.

Each exchange includes:

- Question
 - Transcript
 - Audio metadata
 - Evaluation
 - Timestamp
 - AI reasoning metadata
 - Scores
-

5.8 Evaluation

Each answer is evaluated independently.

Metrics include:

- Technical Accuracy
- Communication
- Confidence
- Completeness
- Problem Solving
- Relevance

Scores accumulate throughout the interview.

5.9 Report Generation

Generate a final report containing:

Overall Score

Skill-wise Breakdown

Strengths

Weaknesses

Suggested Improvements

Recommended Learning Topics

Interview Summary

Exportable PDF

6. Non-Functional Requirements

Performance

- Resume parsing under 10 seconds.
- Interview latency under 3 seconds per response.
- Report generation under 15 seconds.

Scalability

- Stateless API design.
- Modular services.
- Independent AI providers.

Reliability

- Recoverable interview sessions.
- Proper exception handling.
- Structured logging.

Security

- File validation.
- Secure temporary storage.
- Input sanitization.
- Rate limiting.

Maintainability

- Layered architecture.
- Service-oriented design.
- Repository abstraction where appropriate.
- Configuration-driven services.

7. Out of Scope (Version 1)

The following are intentionally excluded:

- User authentication
- Payment integration
- Admin dashboard
- Multi-language interviews
- Video interviews
- Real-time multiplayer
- Company analytics
- Recruiter portal
- Scheduling
- Interview history
- Email notifications

These will be considered in future versions.

8. High-Level Backend Services

- Upload Service
- Resume Parser
- Job Description Parser
- Candidate Profile Service
- Job Profile Service
- Interview Blueprint Service
- Interview Session Service

- AI Conversation Service
- Speech-to-Text Service
- Text-to-Speech Service
- Evaluation Service
- Report Service

Each service has a single responsibility and communicates through well-defined interfaces.

9. Core Domain Entity

The Interview Session is the central entity of the application.

Everything is linked to a single Interview Session:

- Uploaded Resume
- Uploaded Job Description
- Candidate Profile
- Job Profile
- Interview Blueprint
- Conversation History
- Evaluation Scores
- Final Report

This entity acts as the aggregate root for the system.

10. Success Metrics

Version 1 is considered successful if a user can:

1. Upload a resume.
2. Upload a job description.
3. Complete an AI voice interview.
4. Receive adaptive questioning.
5. Obtain a detailed performance report.

without requiring authentication or manual intervention.

11. Engineering Principles

- Simplicity over premature complexity.
- Clean separation of concerns.
- Production-ready code quality.
- Extensible architecture.
- Testable services.
- Clear domain boundaries.

- AI provider abstraction.
- Database-first design.
- Comprehensive logging and error handling.